Step 1: Interaction Mechanics & Input Paradigm Mapping

1. **User Flow Selection:**
    a. Going off of my interviews, it looks like the offboarding process could use some work. We identified several aspects that could use some more information, as well as a smoother process. I will be specifically referencing my second interview where HR still needs access to the user's email account and OneDrive. I will be designing an offboarding wizard that should guide the admin through the process. It should handle license removal and warnings, as well as OneDrive access. This also helps with the issue of remembering the which user the admin is currently working on, since the wizard will show the selected user at the top of the new window.
    b. There should be a button on the selected user's page that only the admin can see to bring up the "Offboarding Wizard". Once opened, the process should be as follows:
        i. Tabs at the top, starting with email. Several options including: "selected user's email is no longer needed in any capacity", "selected user's email will be needed for records", "forward selected user's emails to another specific user", and "forward selected user's emails to a group", any number of these should be selectable at the same time. The goal is to take care of the proper steps behind the scenes once the selected options are chosen. If the selected user's email is still needed, it will be converted into a shared mailbox or distribution list automatically. This will reflect in the confirmation tab.
        ii. Second tab, OneDrive. Options include: "Selected user's OneDrive is no longer needed", "selected user's OneDrive is needed for records", and "Give another user access to selected user's OneDrive", any number of these should be selectable at the same time.
        iii. Third tab, confirmation. This tab will clearly show the admin what is about to happen after they hit "confirm" at the bottom. It should show what processes will take place in the back end, and how these changes will affect the selected user. The confirmation tab will show things like "selected user will no longer have access to their account", "selected user's email will be forwarded to all users in [group]", "selected user's email will be saved as a .PST file for records, place this in a safe place", "selected user's OneDrive contents will be added to [user]'s OneDrive", "all of selected user's licenses will be removed, you will have 30 days to reapply licenses before all contents will be

removed permanently (the licenses being removed are [license 1….])", "click on the box below to verify these changes". The changes that happen in the background include things like converting their mailbox into a shared mailbox or distribution list. The confirmation tab should also show what each license that is being removed does, since that won't take much more effort. This will make it so that the admin can see at a glance what the user had access to. It will also start downloading any files if the admin selected any option to save files for records.

iv. Fourth tab, progress. This will not be a selectable tab, but instead the wizard will navigate to this tab after the admin selects the "confirm" button on the confirmation tab. In this tab it will show a live tracker showing the current state of what is going on with the selected user's account. If the selected user's mailbox has to be converted into a shared mailbox, then there will be several elements describing that the mailbox is being converted, that the license is being removed, and that any specified other users are being given access. Rather than a progress bar, I think an approximate time of completion would be better suited here. It will likely have to update every 30 seconds or so.

v. If multiple users need to be selected in order to assign privileges, the admin will be able to select multiple users from the search bar when selecting that specific option.

c. The purpose of this wizard is to bring all of the important elements into one place that is easy to navigate.

2. **Input Mechanics Analysis:**

a. Since the purpose is to make a wizard that will simplify the process and make it easier for admins to navigate quickly, the wizard should be short and simple. The entire window will be navigable with the keyboard, but a mouse will also work. There may be occasional times when the admin is required to navigate through a list of other users in order to assign privileges. In this case, the admin will have the ability to either search by typing in that user's name, or they can navigate through a list using the arrow keys.

b. As far as accessibility goes, tab targeting is a must. The current admin page is abysmal with its tab targeting, so I intend for the wizard to be a modal dialog with a focus trap. This means that we can reset the tab target priority to the top of the new window and not have to worry about the existing interface. Pressing escape will prompt the admin to close the wizard window and return the focus to the wizard button. The admin will be able to navigate through the

tabs from the top, and then it will deposit their selector to the contents of the currently selected tab. The arrow keys can also navigate through the tabs using left and right, with the up and down arrow navigating through the contents of each tab. There shouldn't be more than about 5 or 6 options per tab of the wizard, so navigating it should be much faster.

c. Screen reader should be able to describe all of the options as they are focused on. It can read off the main tab function whenever the admin selects that specific tab, and then whenever the admin selects an option inside of the tab the screen reader can read out what that option is. The screen reader will also announce a clear message when the admin goes to select the confirmation button. On the progress tab, the screen reader will announce the initial start of the process, and then whenever there is an update it will announce if there are any tasks left. That means after completing a task it should announce something like "assigning selected users access to initial user's mailbox". When the wizard finishes all tasks, the screen reader will announce that it has completed the task.

d. Other devices should be able to use the wizard with a touch screen if necessary since the selector boxes will be large enough to use easily. This is an aspect that is actually quite nice to have. Letting go of an employee isn't always something that is a planned event, so designing an interface that works well on a mobile device is a good way to look out for any unintended times when the admin might have to remove access from a specific user.

3. Fitts' Law Target Sizing:

a. I will be using Google Material Design for my guidelines on element size and padding since it uses a reasonably large size of 48x48dp with at least 8dp of spacing between elements. This means that I can also have a floor of 44x44dp should elements need to be smaller.

b. In order to uphold the 48x48dp seize constraint for my tabs and checkboxes for the wizard, I will be making the entirety of the text for each option function as a clickable are. This will mean that even if the box itself is on the smaller size, the clickable area will far exceed the 48x48dp requirement that I have set. This feature will extent to both the clickable options inside of the tabs, as well as the tabs themselves.

c. A few of my options in the wizard have the admin selecting additional users to give access to. Those options notoriously have quite small 'x's on them in order to remove the selected users after they are added. This makes it fairly difficult to click on if you mistakenly add a user who shouldn't have been added. In order to remediate this, I intend for that field to always be a

navigable list. If you select “Joe” and “Stacy” to be targets to give access to; the wizard will show a box containing their names. Clicking on that box will bring up a larger menu where the admin can navigate all of the users in the same company, just like how they selected Joe and Stacy to begin with, and the admin can clearly see a checkbox next to their names. Clicking anywhere on either of their names will function as a way to remove them from the list.

d. I plan on following the guideline for the element spacing for each individual checkbox in the wizard, meaning that each selectable element should be 8px apart in order to avoid accidentally selecting an unintended option.

e. The confirm and back buttons will both sit at the bottom of each tab, with the confirm button being on the right, and the back button on the left. The button position will never change from tab to tab. The confirm button will instead be a “next” button until the admin is on the confirmation tab. The back button will be greyed out when on the first tab. When the confirm button is pressed, there will be another popup box asking if the admin is sure, and explaining if any of the options are not reversable. This has the function of preventing the admin from accidentally double clicking on the next button, while also working as a guard to prevent the admin from clicking “confirm” when they aren’t ready. That pop up window will have 2 more buttons for “Confirm” and “Cancel”. The focus will automatically be set to cancel, so the user has to make the conscious decision to confirm. The buttons will far exceed the 48x48dp size requirement that I have set for the rest of the wizard since this is an important decision (somewhere in the ballpark of 120x56px).

f. The Math:

i. $MT = a + b \times \log_2(D/W + 1)$

- MT = movement time, how long it takes to hit the target
- D = distance from where the cursor or finger starts to the center of the target
- W = width of the target, measured along the direction of movement
- a and b = constants that depend on the device and person
- $\log_2(D/W + 1)$ = the Index of Difficulty (ID), measured in bits

ii. For reference, I will be comparing my proposed solution against the existing corresponding tab inside of the Office admin portal. I used the developer tools to find the exact measurements and positions.

- The mail button is 44.9x44px.
  a. $x = 325.41 + (44.9 \div 2) = 347.86$
  b. $y = 331.20 + (44 \div 2) = 353.20$

- The convert to shared mailbox button is 163.14x19.2px.
    a. x = 312 + (163.14 ÷ 2) = 393.57
    b. y = 601.70 + (19.2 ÷ 2) = 611.30
- Δx = 45.71 and Δy = 258.10
- Doing some Pythagorean theorem we get (45.71)^2 + (258.1)^2 = D^2
    a. D = 262.1px
- Width = 19.2 since we are moving from top to bottom, so that is the direction of travel for the shared mailbox button.
- ID = $\log_2(262.1 \div 19.2 + 1) = \log_2(14.65) \approx 3.87$
- Index of Difficulty = 3.87 bits.
- Even if I just change the convert to shared mailbox button to the proper 48px tall, it changes the index of difficulty to 2.69 bits. That is about a 30% reduction to the difficulty.

iii. My Version Math

- This was performed on my offboarding wizard after creating it in order to compare results.
- My email tab is 68.7x48px
    a. X = 422.36
    b. Y = 166
- My save records button is 756x72px
    a. X = 768
    b. Y = 285.2
- The rest of the numbers
    a. Δx = 345.64
    b. Δy = 119.2
    c. D = 365.6px
    d. W = 72px (going from top down, so height of records box)
- ID = 2.6 bits
    a. Comparing my 2.6 bits to Microsoft's 3.87 bits, even though my button is about 100px farther away for the mouse to travel, it is still 33% easier to use due to the size of the button.

g. Edge-case hardware

i. Since one of the intended uses of the wizard is for mobile users, a design will have to be implemented that translates well for smaller

screens. The “back” and “next” button will still be at the bottom of the screen on either side. The tabs will be about 1/3rd of the way down from the top, allowing room to display the current selected user as well as the button to close the window. The selectable options inside of each tab will go below the tabs, there will be slightly more space between each element to allow for larger fingers (roughly 12px vs the original 8px). Clicking on any element that requires searching through users will pop up the device’s default keyboard and move the selected element above that so the admin can still see what they are working on. The back and next buttons will be hidden until the keyboard is closed. The screen reader should function the same as it will on the desktop version, reading out each option as focus is shifted to it. To take advantage of gesture controls for phones, the admin should have the option of swiping to navigate through the tabs so they don’t have to reach for the top of the screen.

ii. Voice control should be supported for users who cannot use physical controls, so all of the selectable fields should be labeled correctly corresponding to their intended purpose.

iii. Laptops with touch screens should function similarly to desktops, but they should be allowed to use the swipe controls to navigate between tabs similar to phones.

iv. Admins that only have a keyboard should be able to navigate through the entire wizard with one, having a focus ring around any selected elements to help with this.

Step 2 High-Fidelity Prototyping & Micro-Interaction Design

1. **Interactive Prototype Creation**
    - **a.** In another file.
2. **State Transition Definitions**
    - **a. Offboarding Wizard launch button on existing menu**
        - **i.** Default/Resting
            - Button is red (#C42B1C) with a slightly lighter red border around it (#E35A4D), to distinguish it apart from other menu items
        - **ii.** Hover/Pointer-Over
            - Tooltip text “Start offboarding for this employee”
            - Color changes to #B02718
        - **iii.** Focus

- A 2px white outline around the item, offset by 2px. Shown with ":focus-visible"

iv. Active/Pressed
- Button appears to be pressed in, new window is opening
- Color changes to #9E2216

v. Disabled
- Button is greyed out (#A3A3A3). Applicable when viewing the account of an employee already terminated. In this case, the button should be accompanied by a message such as "user was offboarded on XX/XX/XXXX" so that the admin knows when they were offboarded.

vi. Loading/System Processing
- Button will change to pressed state within 100ms, becoming unclickable (and gaining: aria-busy="true"). After 300ms it changes its contents to a spinning loading ring.

vii. Error/Validation Alert
- Popup will appear stating that the wizard is unavailable.
- Popup needs to inform admin what exactly caused the wizard to fail.
- The launch button will be greyed out and unclickable unless the page is reloaded.

b. Tabs (Email, OneDrive, Confirmation, Progress)

i. Default/Resting
- The currently active tab will have an underlined accent below the text signifying which tab is currently selected (#8CC2FF)

ii. Hover/Pointer-Over
- Faint blue accent appears below hovered-over tab indicating that you intend to switch, nothing happens when hovering over currently accessed tab

iii. Focus
- A 2px white outline around the item, offset by 2px. Shown with ":focus-visible"

iv. Active/Pressed
- The accent will appear if it is not already on that tab, otherwise nothing happens.

v. Disabled

- If the user does not have a Mail or OneDrive, the tab is still accessible, but the contents specify that the user does not have one. Only the Progress tab is ever disabled.

vi. Loading/System Processing

- The tab will be accented, but the contents area will instead have a spinning/loading circle signifying that it is loading.

vii. Error/Validation Alert

- If there is an error navigating tabs then the wizard has experience some kind of failure. A popup will appear explaining that the wizard has encountered an error and that the admin should try to relaunch the wizard to fix the issue.

c. Option rows (checkbox + labels)

i. Default/Resting

- The box will have a slightly different background of #2F2F2F to signify that they are available, but not selected.

ii. Hover/Pointer-Over

- Mouse pointer changes to reflect that you can click on the box

iii. Focus

- A 2px white outline around the item, offset by 2px. Shown with ":focus-visible"

iv. Active/Pressed

- The box will darken slightly at the moment of click, indicating that it has registered the user clicking on it
- The checkbox will be filled in with a check mark upon release of click.

v. Disabled

- Both the option and checkbox will be greyed out (#A3A3A3)
- There will be a visible reason for the option being disabled written below the disabled option

vi. Loading/System Processing

- Since the actual processing happens during tab change or after the confirmation button is pressed, there shouldn't be any loading happening when a button is checked.

vii. Error/Validation Alert

- An error will prevent the admin from changing tabs if there is additional information required for a selection. This will be

indicated by outlining the section that requires more information and adding a red (#FFB3AD) label above the box explaining what additional information is required.

d. User/group picker (for certain options)

i. Default/Resting
- The text box will read placeholder text of “Search users…”

ii. Hover/Pointer-Over
- Mouse pointer changes to reflect that you can click on the box

iii. Focus
- A 2px white outline around the item, offset by 2px. Shown with “:focus-visible”

iv. Active/Pressed
- A searchable menu appears listing all other applicable users for this selection
- The users appear immediately and are later checked in the background. If the selected account fails a check, it is removed from the list and the admin is given the reason such as “that account has been disabled and cannot be given access”

v. Disabled
- Not applicable. If this option is valid, then this box must also be valid.

vi. Loading/System Processing
- A small spinning circle will take the place of the list of all other users while it is loading them in.

vii. Error/Validation Alert
- If there is an error loading other users to assign privileges to: there will be a new box that appears inside of the list that says “could not retrieve other users” followed by a “retry” button.

e. Back Button

i. Default/Resting
- Disabled out on mail tab (#A3A3A3), normal on all others.

ii. Hover/Pointer-Over
- Back button changes to a slightly lighter grey indicating that you are hovering over it

iii. Focus
- A 2px white outline around the item, offset by 2px. Shown with ":focus-visible"

iv. Active/Pressed
- Button will change to pressed state within 100ms, becoming unclickable (and gaining: aria-busy="true"). After 300ms it changes its contents to a spinning loading ring.

v. Disabled
- The box will be greyed out (#A3A3A3)

vi. Loading/System Processing
- Loading circle if the button press takes longer than 300ms to work

vii. Error/Validation Alert
- Not Applicable

f. Next/Confirm Button

i. Default/Resting
- A blue button that stands out from the rest (#195CC4)

ii. Hover/Pointer-Over
- Button color changes to #144A9E

iii. Focus
- A 2px white outline around the item, offset by 2px. Shown with ":focus-visible"

iv. Active/Pressed
- The button becomes a darker color signifying that it was pressed (#0F3A7D)
- Button will change to pressed state within 100ms, becoming unclickable (and gaining: aria-busy="true"). After 300ms it changes its contents to a spinning loading ring.

v. Disabled
- The button will not be clickable while loading into the next tab, or while loading into the confirmation window.

vi. Loading/System Processing
- The button will change its contents to a loading circle after 300ms

vii. Error/Validation Alert
- If there is no change when the admin click on confirm after 10 seconds of loading, then the wizard needs to notify the

admin that there has been an error via a line of red text added below the button. The message should read something like “Couldn’t continue, check your connection and try again”

g. Verification checkbox (on the confirmation tab)

i. Default/Resting
- Unchecked box asking admin to verify confirmation of the process

ii. Hover/Pointer-Over
- Mouse pointer changes to indicate that you can click on it

iii. Focus
- A 2px white outline around the item, offset by 2px. Shown with “:focus-visible”

iv. Active/Pressed
- The checkbox becomes filled with a check mark

v. Disabled
- Not Applicable

vi. Loading/System Processing
- Not Applicable

vii. Error/Validation Alert
- A red (#FFB3AD) message will appear below the checkbox explaining that the admin needs to confirm that they are going to alter the account in a way that may not be recoverable.

h. Confirmation pop-up

i. Default/Resting
- Box will not be visible

ii. Hover/Pointer-Over
- Buttons change color slightly. Cancel becomes more grey, Confirm becomes more blue, like the Next button.

iii. Focus
- The buttons inside can be focused, where they will gain a 2px white outline around the item, offset by 2px. Shown with “:focus-visible”
-

iv. Active/Pressed

- Either the confirmation window will close and the admin will be at the confirmation tab, or it will close and the admin will be at the progress tab.

v. Disabled
- Not applicable

vi. Loading/System Processing
- If for some reason the popup doesn't load right away, there should be a placeholder loading circle signifying that the system is working.

vii. Error/Validation Alert
- If there is an error proceeding to the next step after the admin click on "confirm" then the wizard will give a description in a popup describing what went wrong and provide the admin with a "retry" button.

i. Close Wizard Button

i. Default/Resting
- 'X' Button at the top right

ii. Hover/Pointer-Over
- Mouse pointer changes to indicate that you can click on it

iii. Focus
- A 2px white outline around the item, offset by 2px. Shown with ":focus-visible"

iv. Active/Pressed
- The wizard will prompt the admin if they want to discard changes (only if changes have been made) and close, with "No" being the default selected option.

v. Disabled
- Not applicable

vi. Loading/System Processing
- The window will get a faint white overlay, like other windows applications that are processing their closing.

vii. Error/Validation Alert
- The admin portal will add red (#FFB3AD) text next to the launch wizard button explaining that it encountered an error when trying to close the wizard.

j. Progress task list

i. Default/Resting

- Inaccessible tab that can only be seen after confirming on the confirmation tab and popup.
- Descriptions of each line item appear below each item
- Screen readers learn about progress through: “aria-live announcements”

ii. Hover/Pointer-Over
- Not Applicable

iii. Focus
- Not Applicable

iv. Active/Pressed
- Not Applicable

v. Disabled
- Not Applicable

vi. Loading/System Processing
- There will be a loading circle beside each actively working list item. It will change to a check mark when completed

vii. Error/Validation Alert
- A red error message will appear if the wizard failed to perform a task. Once all tasks are either completed or failed, there will be a popup describing what went wrong and how to resolve the issue. A popup is used here because the wizard has ran its full process and has failed somewhere along the line, so the admin needs to be alerted that something went wrong.

k. Animations – These affect all prior line items:

i. Hover color changes: 150ms ease-out
ii. Tab underline sliding to the new tab: 250ms ease-in-out
iii. Checkbox check appearing: 200ms ease-out
iv. Popup fading in: 200ms ease-out
v. Pressed state: instant (under 100ms)
vi. If the user has the “reduced motion” setting disabled in their OS then all animations are disabled

Step 3: Accessibility Architecture & WCAG Specification

1. **Semantic HTML Markup**
   a. Referencing my code here, I only have 1 page that handles everything.

- The <head> at the top handles the meta data as well as linking the stylesheet.
- The body holds all of the contents of the page. The only visible elements are the “example user” name and the “launch wizard” button. Those are contained inside of the <main> element
- After the <main> element, we get into the dialog box. This is where every element of the wizard resides. I wanted it to pop out of the existing page so that it would draw the admin’s attention and reset the page focus.
- At the top of the dialog box is another header, containing a simple <h2> followed by a <p>. This just shows that you are using the “Offboarding Wizard” and tells you which user you are modifying. There is also an ‘X’ button for closing the wizard in here.
- Next up is a that contains every tab for the wizard.
- Following the we start the <form> tag. This is because every selection of the wizard belongs inside of this form. It collects all the inputs and gathers them together.
- Each tab is divided into its own <section> tag to divide them out.
- Inside of the email and onedrive <sections>, the tab is placed inside of a <fieldset> tag that contains all of the choices for the admin. The confirmation and progress tabs don’t have this because they are a list of items and not section with selectable options.
- At the top of the <fieldset> is a <legend> tag that describes what the tab is designed to handle.
- Every <input> for the options is wrapped inside of a <label> tag to make the entire text box clickable.
- Specific options require text input, those inputs are placed inside of a and pull a list of users that I assign manually.
- After the last “Progress” tab, I close the <form> and add the <datalists> containing users and groups
- After the <datalists> comes the <footer> that contains the back and next buttons.
- After the <footer>, I finally close out the <dialog> of the wizard.
- Finally comes 2 more <dialogs> that contain the 2 possible popups: The “discard changes?” box that shows when the admin clicks on the ‘X’ at the top, and the confirmation dialog box that appears at the end to confirm the process.

- Then we close it all out by adding the <script> tag and closing out the <body> and <html> tags.

2. **ARIA & Screen Reader Hooks**
    a. I'll go back over my code here and reference specifically the ARIA elements.
        - The first and most often used ARIA tag is aria-describedby. This adds a bit more information to be read after the name. I use it for my error messages so that if someone is using a screen reader, they are given additional information on why something failed or is disabled.
        - The next tag is aria-labelledby. That gives each section a name to go by. This gives screen readers more information to tell the admins when they arrive at the tab.
        - Aria-label is next. This names elements with no other visible label. Things like the 'X' button, the picker fields, and the retry button.
        - Aria-selected is used inside of the tab list to show which tab is currently selected. This also drives my css so that they can't disagree.
        - Aria-controls helps to wire up control to the element it is on. It is initially linked in the section where I populate the tabs, and then later it is referenced in the tab's section. This is to connect the tab to its section. The connection is a two-way street though. The tab controls the panel, the "labelledby" points back to the tab's id.
        - At the bottom of the wizard I add aria-live="polite". This will update screen readers when something updates on the page, but only after whatever message is currently being read.
        - Throughout the wizard I add several "alerts" that prevent the admin from continuing unless the conditions are satisfied (done via css). This uses an aria-live="assertive" announcement that reads off its message immediately.
        - Aria-disabled is used on the elements that I want to keep disabled, but they still need to be focusable for the screen reader.
        - I know that I don't use aria-expanded, but the only place I could really use that would be on the pickers, and apparently aria-expanded doesn't allow for checkboxes. I just used aria-controls instead.
3. **Visual Accessibility**
    a. In the initial design of my wizard I had to keep a few things in mind: I wanted to make the design WCAG AA compliant (AAA when possible), and I wanted to keep the design looking similar to the dark theme of the existing admin portal. For the most part, my page is white text on a black background, like

the admin portal. Where my design differs from the existing admin portal is by increasing the size and contrast of all elements. This is to address the 52 contrast errors that I found during phase 1.

b. Using #F5F5F5 for my page and wizard text, and #121212 and #252525 for the page and wizard backgrounds respectively lands me at a contrast of 17.2:1 and 14.1:1. My worst text contrast is actually for my disabled back and launch buttons (#A3A3A3 on #333333) with a ratio of 5:1, still falling within compliance.

c. One of the most important contrast elements is the focus indicator. The indicator is 2px of solid white with a 2px offset. This means that the contrast is at worst 13:1.

d. My error messages are a light red (#FFB3AD) and have a contrast ratio of 9:1, with an additional red outline on the picker fields.

e. My text is 13px at its smallest, but on average it is around 15px.

f. My css disables all transition animations if the user has the “reduced motion” setting enabled in their OS.

g. My tab underline has a ratio of 8.2:1, and the progress icons have a ratio of at least 5.3:1

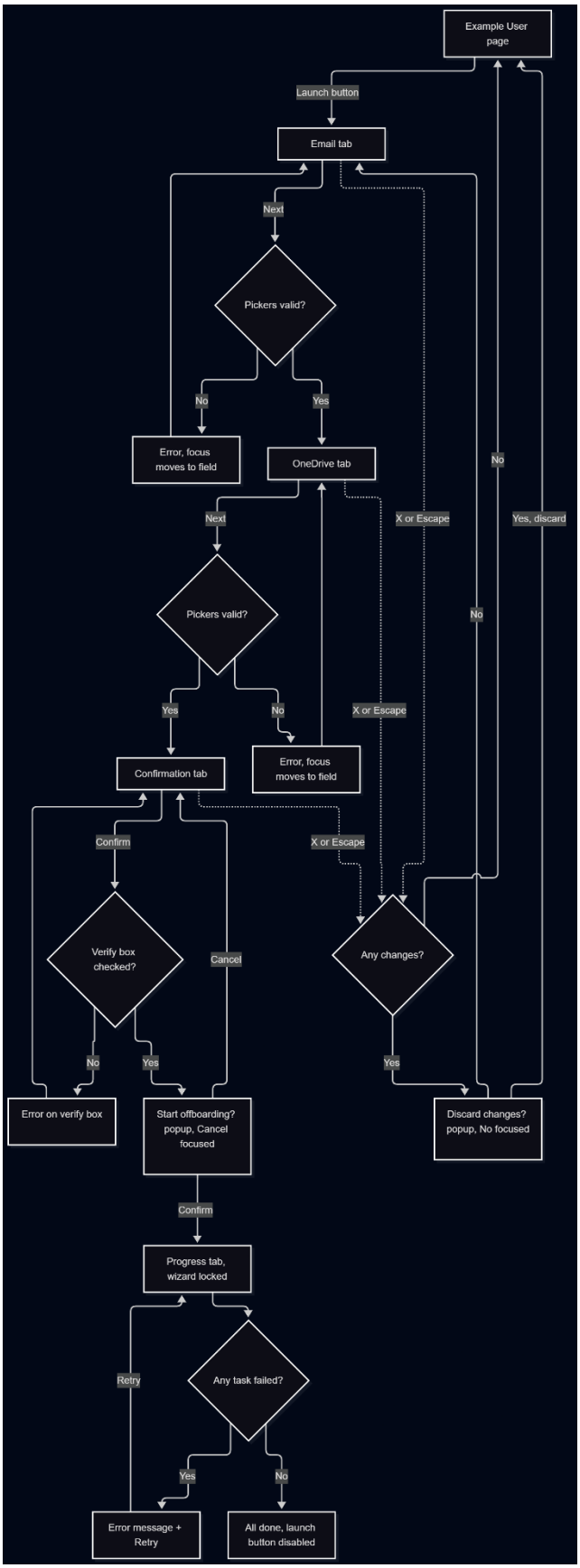
h. All of my errors pertaining to the pickers have the error text, plus they outline the problem area. This means that even if the admin is colorblind, they can still see what is going on.

i. The one issue I was going to have with my design is the “next” button would only have a contrast ratio of 2.5:1, so I gave it a light blue border that has a ratio of 8.2:1. This makes it so the button can remain a darker color, but still stand out from the background.

j. All of my ratios were checked with the WebAIM contrast checker.

Step 4 Additions:

Image generated using mermaid.live.


Example User page
Launch button
Email tab
Next
Pickers valid?
No
Yes
Error, focus moves to field
OneDrive tab
Next
X or Escape
Yes, discard
No
Pickers valid?
No
Yes
No
X or Escape
Confirmation tab
Error, focus moves to field
Confirm
X or Escape
Verify box checked?
Cancel
Any changes?
No
Yes
Yes
Error on verify box
Start offboarding? popup, Cancel focused
Discard changes? popup, No focused
Confirm
Progress tab, wizard locked
Retry
Any task failed?
Yes
No
Error message + Retry
All done, launch button disabled

Post-Mortem Reflection (Metacognition)

1. **How did explicitly defining every component state (Hover, Focus, Active, Loading, Error) during the specification phase prevent dynamic usability bugs that static wireframes completely fail to address?**
    a. The static wireframe would show a list of checkboxes, but only the state matrix forced me to decide what happens when two conflicting boxes are checked. Rather than simply deciding that my boxes were disabled, I had to make the conscious choice to use aria-disabled so that the elements could still be focused on and read with the screen reader. I think a big thing for me was that a wireframe can hide a lot of errors and edge cases. Defining the component states made me think about every way that they could fail, and build around them.
    b. Deciding where the admin's focus goes is another interesting thing that I hadn't put much thought into until this. Being able to control where the focus starts and ends is a minor thing for most people, but is a huge quality of life change for users who navigate using only a keyboard. It also meant that I could do things like setting the focus to the "cancel" option in the last popup of the wizard since the admin needs to make a conscious choice to click "confirm".
2. **Describe a scenario in your prototype where system latency or asynchronous data processing required specialized micro-interaction feedback (e.g., loading skeletons or aria-live announcements). How does your design keep the user informed and prevent repeated submission errors?**
    a. My design incorporates several key elements that are intended to keep the admin informed. I created a progress tracker that follows along with each task as they are handled in the background, as opposed to Microsoft's "wait 24-48 hours" message. This tracker not only shows list items of every task, but it also shows the loading indicator to tell the admin that it is actively working on that task. I prevent repeated submission errors by having multiple verification points throughout the wizard as well as disabling the wizard button entirely after the process has already been completed. The admin has to check the "agree" box, then they have to click "confirm", then they have to click "confirm" again in the new dialog box. Once the employee has been offboarded; I also add a message next to the launch button on the admin page that it was disabled on whatever date.
    b. I designed a failure case in my progress tab. This was initially to test it out and see how it looked in the case of an actual failure. Adding a "retry" button and

message to that error added a handy way of keeping the admin informed while also allowing them to start just that process again. This means that the admin doesn't have to open the wizard again and select all of their options all over. They can just hit retry right there in that tab and continue where they left off.

c. I use aria-live polite for most of the updates throughout the wizard, but in order to bring the admin's attention to some kind of error or missing field, I use the "alert" role. This functions the same way as the aria-live "assertive" tag. I had to account for this because admins who use a screen reader shouldn't encounter an error and have to wait on the reader to finish whatever piece of text it was on before in order to hear about the error. During the progress tab, the polite log reads out each task as it changes to keep the admin informed of the process.